

1. Circular Shift Let A be an array of n distinct integers which are sorted into increasing order and then circularly shifted by k positions to the right. For instance, $[3, 4, 8, 11, 15, 18]$ is sorted and shifted by $k = 3$ into $[11, 15, 18, 3, 4, 8]$.

Given such an array which has been shifted by an unknown k , describe a logarithmic-time algorithm that determines the value of $k \bmod n$.

The naive solution is to step through the array until the index i is found such that $a_i > a_{i+1}$. Given that $k \bmod n \neq 0$, such an index must exist and we have $k = i$. In fact, we also have that k is the earliest index for which $a_1 > a_{k+1}$ by the monotonicity of A before its shift.

A logarithmic solution is a simple binary search for the earliest such index: $i = \lfloor \frac{n}{s} \rfloor$, if $a_{i+1} > a_1$, then we must have $k > i$ and we can search $A[i + 1 : n]$. Otherwise, search $A[1, i]$.

2. Inversions Given a permutation A of $\{1, 2, \dots, n\}$ and two indices $i < j$ we say that i and j are inverted if $a_i > a_j$. For example, $A = [5, 3, 4, 1, 2]$ has eight inverted pairs: $(1, 2)$, $(1, 3)$, $(1, 4)$, $(1, 5)$, $(2, 4)$, $(2, 5)$, $(3, 4)$ and $(3, 5)$.

Describe an algorithm which counts the number of inverted index-pairs in a given permutation with time complexity $\mathcal{O}(n \log n)$.

The naive solution uses two nested loops which iterate over the permutation counting the inversions— $\mathcal{O}(n^2)$.

Mergesort can be modified to count the number of inversions recursively in the correct complexity. In particular, the merge subroutine:

Algorithm `merge_and_count( $L, R$ )`

```

 $i \leftarrow 1, \quad j \leftarrow 1, \quad \text{count} \leftarrow 0$ 
while  $i + j < n + 1$  do
    if  $l_i < r_j$  then
         $b_{i+j-1} \leftarrow l_i$ 
         $i \leftarrow i + 1$ 
    else
         $b_{i+j-1} \leftarrow r_j$ 
         $j \leftarrow j + 1$ 
         $\text{count} \leftarrow \text{count} + \text{len}(L) - i + 1$ 
    end if
end while
return  $(B, \text{count})$ 

```

When an element from the right list is added to B , it is placed before $\text{len}(L) - i + 1$ many elements from the left list which implies the existence of $\text{len}(L) - i + 1$ many inverted pairs in the original permutation. Summing these `count` values as we ascend the recursion tree gives the total number of inversions.

3. Twisted cable We have a long data cable with n individual wires exposed on either end; each wire on the left end is connected to exactly one wire on the right end. We need to find out which wires map to which in order to transmit data efficiently, but the cable has been twisted, so this mapping is not obvious.

We can use the following operations:

- connect electricity to a given wire on the left end,
- disconnect electricity from a given wire on the left end and
- measure electricity on a given wire on the right end.

Design an algorithm to find out which wire is connected to which while minimizing the number of these operations.

11th tutorial (continued)

We are given two sets L and R of n wires each—left and right side respectively. A divide-and-conquer approach works well on this problem:

- Split L into two halves L_1 and L_2 .
- Connect each wire in L_1 and disconnect those in L_2 .
- Measure electricity on all the wires in R . Those that test positive are placed in R_1 and those negative in R_2 .

The problem has now been reduced to two subproblems with half the size. When we get sets of size 1, the problem is solved.

The complexity of this recursive procedure is given by the recurrence $T(n) = 2T(n/2) + \Theta(n)$, $T(1) = 1$ the solution to which is $T(n) = \Theta(n \log n)$ by the Master theorem.

An alternate, non-recursive solution

Let $k = \lceil \log n \rceil$ so that each wire in L may be assigned a unique k -bit binary code.

In each phase, we analyze one bit: in phase j , we connect electricity only to the wires with the j -th bit equal to one; the wires in R which have electricity have their j -th bit also set to one. By the end, each wire in R has the same code as their corresponding wire in L .

In each of the $\Theta(\log n)$ phases, we perform $\Theta(n)$ actions. Therefore, the complexity is $\Theta(n \log n)$.

4. Nuts And Bolts At the input, we are given n nuts and n bolts of different sizes. For each bolt, there is at least one suitable nut but we can only compare one nut to one bolt at a time. Doing so tells us that either: the nut is too small, the nut is too large or the nut and bolt fit together.

Describe an algorithm which pairs nuts and bolts as quickly as possible. A randomized solution with average time $O(n \log n)$ is enough.

We have two sets $N, B \in \mathbb{Z}^n$ representing our nuts and bolts respectively along with a comparison function $N \times B \mapsto \{-1, 0, 1\}$:

$$\text{comp}(\nu, \beta) = \begin{cases} -1, & \text{if } \nu < \beta \\ 0, & \text{if } \nu = \beta \\ 1, & \text{if } \nu > \beta \end{cases}$$

We begin by picking one bolt $\beta^* \in B$ at random, compare it to each nut and group the nuts by output:

$$N_{-1} = \{\nu: \text{comp}(\nu, \beta^*) = -1\} \quad N_0 = \{\nu: \text{comp}(\nu, \beta^*) = 0\} \quad N_1 = \{\nu: \text{comp}(\nu, \beta^*) = 1\}.$$

Next, we pick an arbitrary nut ν^* from N_0 , compare it against every bolt and form similar groups of bolts:

$$B_{-1} = \{\beta: \text{comp}(\nu^*, \beta) = -1\} \quad B_0 = \{\beta: \text{comp}(\nu^*, \beta) = 0\} \quad B_1 = \{\beta: \text{comp}(\nu^*, \beta) = 1\}.$$

Clearly $|N_{-1}| = |B_1|$, $|N_0| = |B_0|$, $|N_1| = |B_{-1}|$ and the problem is divided in exactly $2n$ comparisons— $\Theta(n)$. We also expect $|N_{-1}| \approx |N_1| \approx \frac{n}{2}$ and we may recur on the two sub-problems: (N_{-1}, B_1) and (N_1, B_{-1}) . By the master theorem, the total number of comparisons is $T(n) = 2T(\frac{n}{2}) + \Theta(n) \in \Theta(n \log n)$.